Code Walkthrough: Data Leakage Detection Notebook

A section-by-section, line-by-line explanation of *leakage_detection_combined.ipynb*

How to read this document

Each section matches one code cell of the notebook, in order. For every cell you get: what it is for, the code with line numbers, a table explaining the lines, and the result it produced on the Kaggle dataset. Line numbers refer to lines inside that cell.

The big picture

1. **Setup:** prepare the environment and get the dataset.
2. **Load and audit:** read the data, create the hour attribute, check quality.
3. **Ceiling:** calculate the best accuracy any model could reach.
4. **Split:** lock 20% of the data away as the final test set.
5. **Reproduce:** re-evaluate the article's depth-3 tree.
6. **Tune:** search 80 tree settings with cross-validation.
7. **Attributes:** test External Destination and hour.
8. **Compare:** tree vs Random Forest vs XGBoost; four imbalance strategies.
9. **Threshold:** choose when to raise an alarm (catch ≥ 80% of leaks).
10. **Final test:** open the test set once and measure everything.
11. **Explain:** IF-THEN rules and attribute importance.
12. **Save:** store the model, tables and charts.

One rule runs through the whole notebook: **the test set is never used for any decision**. All choices (settings, attributes, model, threshold) are made with cross-validation on the training data. This is what makes the final results trustworthy.

Glossary

Term	Meaning
DataFrame	A table in pandas: rows are events, columns are attributes.
Feature / attribute	An input the model uses, e.g. Authority or hour.
Label	The answer the model learns to predict: Abnormality (0 = normal, 1 = leakage).
Training set / test set	80% of events used to learn; 20% locked away and used once to measure final performance.
Cross-validation (5-fold)	Splitting the training data into 5 parts; training on 4 and checking on the 5th, five times, then averaging.
Pipeline	A chain of steps (preparation, then model) treated as one object, so preparation learns only from training data.
Imputation	Filling missing values, here with the most frequent value (or the median for hour).
One-hot encoding	Turning a category into 0/1 columns, e.g. Authority = manager becomes Authority_manager = 1.
Class weights	Making errors on the rare class (leakage) count more during training.
Threshold (θ*)	The risk score above which an event raises an alarm.
Recall	Share of real leaks the model catches.
Precision	Share of alarms that are real leaks.
F1	One number balancing precision and recall.
PR-AUC (average precision)	How well the model ranks risky events above normal ones, across all thresholds.
MCC	A balanced score from −1 to 1; 0 means no better than chance.
Data leakage (methodological)	When information from the test data sneaks into training, making results look better than they are.

Python notation used often

Notation	Meaning
`import x / from x import y`	Loads a library or one tool from it.
`x = ...`	Stores a value under a name (a variable).
`def name(...):`	Defines a reusable function.
`for x in ...:`	Repeats the indented lines for each item.
`df['col']`	Selects one column of a table.
`df[cols]`	Selects several columns.
`{key: value}`	A dictionary: a lookup of names to values.
`[a, b, c]`	A list.
`f'...{x}...'`	Text with a value inserted (f-string).
`# comment`	Ignored by Python; a note for humans.
`random_state=42`	Fixes randomness so results are identical every run.
`n_jobs=-1`	Use all processor cores.

0. Setup

Prepares the environment: loads basic tools, detects whether the notebook runs in Google Colab, installs missing packages, gets the dataset, and creates the output folders.

Code

```
# Works in Google Colab and locally. In Colab: installs packages and asks for the CSV if it is missing.
import os, sys, json, shutil, warnings
warnings.filterwarnings('ignore')
DATA = 'data/raw/Data_Leakage_Detection.csv'
# Using Google Drive instead? Mount it and point DATA at the file, e.g.
# from google.colab import drive; drive.mount('/content/drive')
# DATA = '/content/drive/MyDrive/leakage_project/Data_Leakage_Detection.csv'

try:
    import google.colab
    IN_COLAB = True
except ImportError:
    IN_COLAB = False

if IN_COLAB:
    !pip install -q imbalanced-learn xgboost
    if not os.path.exists(DATA):
        os.makedirs('data/raw', exist_ok=True)
        from google.colab import files
        for name in files.upload():            # choose Data_Leakage_Detection.csv
            shutil.move(name, DATA)

os.makedirs('results', exist_ok=True); os.makedirs('models', exist_ok=True)
import pandas as pd, numpy as np, matplotlib.pyplot as plt
import sklearn, imblearn, xgboost
print('dataset found:', os.path.exists(DATA))
print('python', sys.version.split()[0], '| scikit-learn', sklearn.__version__,
      '| imbalanced-learn', imblearn.__version__, '| xgboost', xgboost.__version__)
```

Line by line

Lines	What it does
2	Imports Python's built-in modules: **os** (files and folders), **sys** (Python version), **json** (saving the run record), **shutil** (moving files) and **warnings**.
3	Hides warning messages so the output stays readable. Warnings are not errors; they are notices such as future version changes.
4	**DATA** stores where the CSV file is expected. Every later cell reads the data from this path.
5–7	Comments only (lines starting with # are ignored by Python). They show how to read the file from Google Drive instead.
9–13	**try / except**: tries to import **google.colab**, which exists only inside Colab. If it works, **IN_COLAB = True**; if Python raises **ImportError**, the notebook is running on a normal computer and **IN_COLAB = False**.
15–16	Only in Colab: **!pip install** runs a terminal command that installs **imbalanced-learn** and **xgboost**. The **!** means "run this as a system command"; **-q** means quiet.
17–21	If the CSV is not there yet, it creates the folder **data/raw**, opens Colab's upload button (**files.upload()**), and moves the uploaded file to the expected path.
23	Creates the folders **results** (tables and charts) and **models** (the trained model). **exist_ok=True** means "no error if the folder already exists".
24	Imports the data-science libraries: **pandas** (tables, called DataFrames), **numpy** (numbers and arrays) and **matplotlib** (charts). **pd**, **np** and **plt** are short nicknames.
25	Imports the machine-learning libraries so their version numbers can be printed.
26–28	Prints whether the dataset was found and the version of each library. Recording versions makes the results reproducible.

Lines	What it does

Result: Prints "dataset found: True" and the library versions.

1. Loading the dataset

Reads the CSV into a table, creates the **hour** attribute from the date, and defines which columns are the label and the features.

Code

```
df = pd.read_csv(DATA)

# hour of day from 'date' (two text formats; the day/month order of the
# second one is ambiguous, but the hour is not, so only the hour is used)
s = df['date'].astype('string')
dt = pd.to_datetime(s, format='%m/%d/%Y %H:%M', errors='coerce').fillna(
     pd.to_datetime(s, format='%m-%d-%y %H:%M', errors='coerce'))
df['hour'] = dt.dt.hour

label = 'Abnormality'
features9 = ['Authority', 'Through_pwd', 'Through_pin', 'Through_MFA',
             'Data Modification', 'Confidential Data Access',
             'Confidential File Transfer', 'File Operation', 'Data Sensitivity Level']
ED, HOUR = 'External Destination', 'hour'
ALL = features9 + [ED, HOUR]          # id, date, user, pc are never used as features

print(df.shape)
df.head()
```

Line by line

Lines	What it does
1	**pd.read_csv** reads the CSV file into a pandas DataFrame called **df**: a table with 49,500 rows (events) and 15 columns.
5	Takes the **date** column and treats it as text.
6–7	**pd.to_datetime** converts text into real dates. The file mixes two formats: "2/26/2020 12:43" (**%m/%d/%Y %H:%M**) and "07-10-14 0:54" (**%m-%d-%y %H:%M**). The first line tries the first format; **errors='coerce'** turns failures into empty values instead of crashing. **.fillna(...)** then fills those empty values by trying the second format.
8	**dt.dt.hour** extracts the hour (0–23) and stores it in a new column, **hour**. Only the hour is used because the day/month order of the second format is ambiguous, while the hour is not.
10	**label** names the column the model must predict: **Abnormality** (0 = normal, 1 = leakage).
11–13	**features9** lists the nine attributes used in the reference article.
14	Short names for the two extra attributes: **ED** = External Destination, **HOUR** = hour.
15	**ALL** = the nine attributes plus the two extras: the 11 features of the final model. The columns id, date, user and pc are deliberately left out, because identifiers let a model memorise rows or people instead of learning behaviour.
17–18	Prints the table size and shows the first five rows.

Result: (49500, 16): 49,500 events and 16 columns (15 original + hour).

2. Data audit

Checks data quality before any modelling: missing values, the values of each attribute, and the class balance.

Code

```
print('Missing values:'); print(df.isna().sum(), '\n')
for col in features9 + [ED]:
    print(f'{col:28s} →', df[col].value_counts(dropna=False).to_dict())
print('\nLabel:'); print(df[label].value_counts())
print(df[label].value_counts(normalize=True).round(3))
print('\nUnparsed dates (hour missing):', df[HOUR].isna().sum())
```

Line by line

Lines	What it does
1	**df.isna().sum()** counts the missing values in each column.
2–3	Loops over each attribute and prints how often each value appears (**value_counts**). **dropna=False** also counts missing values. **f'{col:28s}'** pads the name to 28 characters so the output lines up.
4–5	Counts normal and leakage events, first as numbers, then as proportions (**normalize=True**).
6	Counts events whose date could not be read, so their hour is missing.

Result: About 1% missing values per attribute; 34,105 normal (68.9%) and 15,395 leakage (31.1%) events.

2b. Leakage rate by hour

Draws a chart of how often leakage happens at each hour of the day, to see whether the hour carries information.

Code

```
rate = df.groupby(HOUR)[label].mean()
ax = rate.plot(kind='bar', figsize=(10, 3), color='#4C72B0')
ax.axhline(df[label].mean(), color='grey', ls='--', lw=1, label='overall rate')
ax.set_title('Leakage rate by hour of day'); ax.set_ylabel('share labelled leakage'); ax.legend()
plt.savefig('results/leakage_by_hour.png', dpi=150, bbox_inches='tight'); plt.show()
work = df[HOUR].between(9, 17)
print('leakage rate 09:00-17:59:', round(df.loc[work, label].mean(), 3),
      '| outside:', round(df.loc[~work & df[HOUR].notna(), label].mean(), 3))
```

Line by line

Lines	What it does
1	**groupby(HOUR)[label].mean()** groups events by hour and averages the label. Because leakage = 1 and normal = 0, the average is the share of leakage events in that hour.
2	Draws those 24 values as a bar chart.
3	**axhline** adds a dashed horizontal line at the overall leakage rate (31%), for comparison.
4–5	Adds a title, axis label and legend, saves the chart as a PNG in **results/**, and displays it.
6	**between(9, 17)** marks events from 09:00 to 17:59 as working hours (True) and the rest as False.
7–8	Prints the leakage rate inside and outside working hours. **~work** means "not working hours"; **df[HOUR].notna()** excludes events with no hour.

Result: Leakage is about 15.8% during working hours and 40.4% outside them.

3. Performance ceiling

Calculates the highest accuracy any model could possibly reach with a given set of attributes. Events with exactly the same attribute values but different labels cannot be told apart by any model.

Code

```
def ceiling_report(cols, name):
    # missing values are treated as their own category
    key = df[cols].astype(str).fillna('nan').agg('|'.join, axis=1)
    tab = pd.crosstab(key, df[label])
    n = len(df)
    return {'attributes': name, 'combinations': len(tab),
            'majority': round(df[label].value_counts(normalize=True).max(), 4),
            'ceiling Acc*': round(tab.max(axis=1).sum() / n, 4),
            'mixed share c': round(tab[tab.min(axis=1) > 0].sum(axis=1).sum() / n, 4)}

ceil = pd.DataFrame([ceiling_report(features9, 'Nine attributes (article)'),
                     ceiling_report(features9 + [ED], '+ External Destination')])
ceil.to_csv('results/ceiling.csv', index=False)
ceil
```

Line by line

Lines	What it does
1	Defines a function, **ceiling_report**, so the same calculation can run for different attribute sets.
3	Builds a "fingerprint" for each event by joining its attribute values into one text, e.g. "manager|0|0|1|0|0|1|move|low". Missing values become the text "nan", so they count as their own category.
4	**pd.crosstab** builds a table: one row per fingerprint, with the number of normal and leakage events that share it.
6	**combinations** = how many different fingerprints exist.
7	**majority** = accuracy of always predicting the most common class ("normal").
8	**ceiling Acc***: for each fingerprint, the best possible guess is its more frequent label, which gets **max** events right. Adding these up over all fingerprints and dividing by the total gives the best achievable accuracy.
9	**mixed share c**: the share of events whose fingerprint contains both labels (**min > 0**). These events are inherently ambiguous.
11–14	Runs the function for the nine article attributes and for nine + External Destination, saves the table as CSV, and displays it.

Result: Nine attributes: 4,689 combinations, ceiling 0.743, 91.1% of events ambiguous. With External Destination: ceiling 0.779.

4. Split, pipeline and reference tree

The most important cell for a fair evaluation. It splits the data once, defines how the data are prepared, and re-evaluates the article's model.

Code

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier

X, y = df[ALL], df[label]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = {'ap': 'average_precision', 'f1': 'f1', 'recall': 'recall',
           'precision': 'precision', 'acc': 'accuracy'}
print('Train:', X_tr.shape, ' Test:', X_te.shape, '(test set locked)')

def make_prep(cols):
    # categorical attributes: impute + one-hot; hour: impute, kept numeric
    cat = [c for c in cols if c != HOUR]
    steps = [('cat', Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                               ('onehot', OneHotEncoder(handle_unknown='ignore'))]), cat)]
    if HOUR in cols:
        steps.append(('num', SimpleImputer(strategy='median'), [HOUR]))
    return ColumnTransformer(steps, verbose_feature_names_out=False)

def tree_pipe(cols, **params):
    return Pipeline([('prep', make_prep(cols)),
                     ('tree', DecisionTreeClassifier(random_state=42, **params))])

def cv_row(name, model, cols):
    s = cross_validate(model, X_tr[cols], y_tr, cv=cv, scoring=scoring, n_jobs=-1)
    return {'model': name, **{m: round(s[f'test_{m}'].mean(), 4) for m in scoring},
            'f1_std': round(s['test_f1'].std(), 4)}

baseline = tree_pipe(features9, criterion='entropy', max_depth=3)
ref_cv = pd.DataFrame([cv_row('Reference tree (entropy, depth 3)', baseline, features9)])
ref_cv
```

Line by line

Lines	What it does
1–6	Imports scikit-learn tools: splitting and cross-validation, the **Pipeline**, **ColumnTransformer** (different preparation for different columns), **SimpleImputer** (fills missing values), **OneHotEncoder** (turns categories into 0/1 columns) and the **DecisionTreeClassifier**.
8	**X** = the 11 feature columns (the inputs); **y** = the label (the answer).
9	**train_test_split** splits the data **once**: 80% training (**X_tr, y_tr**), 20% test (**X_te, y_te**). **stratify=y** keeps 31% leakage in both parts. **random_state=42** fixes the random shuffle so the split is the same every time. The test set is not touched again until Section 9.
10	**StratifiedKFold** sets up 5-fold cross-validation: the training data are cut into 5 parts; each model trains on 4 and is checked on the 5th, five times. Every comparison in the notebook uses these same folds.
11–12	The five scores measured each time: **average_precision** (PR-AUC, ranking quality), **f1**, **recall**, **precision** and **accuracy**.
15–22	**make_prep** builds the data-preparation step. Text and 0/1 attributes: fill missing values with the most frequent value, then one-hot encode (e.g. Authority becomes five columns such as Authority_manager = 1 or 0). **handle_unknown='ignore'** avoids errors if a value appears only in test data. The hour stays a number; missing hours get the median. Because this is inside the pipeline, it learns only from the training folds, which prevents data leakage.

Lines	What it does
24–26	**tree_pipe** chains preparation + decision tree into one object. ****params** passes settings such as depth through to the tree.
28–31	**cv_row** runs 5-fold cross-validation for one model and returns the average of each score, plus **f1_std**: how much F1 varies between folds (lower = more stable). **n_jobs=-1** uses all processor cores.
33–35	Re-creates the article's model, a tree with **criterion='entropy'** and **max_depth=3**, on the nine attributes, and evaluates it.

Result: Reference tree: accuracy 0.691, recall 0.151. It barely beats always predicting "normal" (0.689).

5. Tuning the tree (nine attributes)

Searches for the best tree settings automatically.

Code

```
param_grid = {'tree__criterion': ['gini', 'entropy'],
              'tree__max_depth': [3, 5, 7, 10, None],
              'tree__min_samples_leaf': [1, 20, 50, 100],
              'tree__class_weight': [None, 'balanced']}
gs9 = GridSearchCV(tree_pipe(features9), param_grid, cv=cv, scoring=scoring, refit='ap', n_jobs=-1)
gs9.fit(X_tr[features9], y_tr)
print('Best:', gs9.best_params_)
res = pd.DataFrame(gs9.cv_results_)
cols = ['param_tree__class_weight', 'param_tree__criterion', 'param_tree__max_depth',
        'param_tree__min_samples_leaf', 'mean_test_ap', 'mean_test_f1', 'mean_test_recall',
        'mean_test_precision', 'mean_test_acc']
res.sort_values('mean_test_ap', ascending=False)[cols].head(8).round(4)
```

Line by line

Lines	What it does
1–4	**param_grid** lists the settings to try. **criterion**: how the tree measures a good split (gini or entropy). **max_depth**: how many questions in a row the tree may ask (None = no limit). **min_samples_leaf**: the minimum number of events in a final group, which stops the tree from memorising tiny groups. **class_weight**: **'balanced'** makes each leakage event count about 2.2 times more in training, so the tree stops ignoring the minority class. 2 × 5 × 4 × 2 = **80 combinations**. The prefix **tree__** tells the pipeline which step a setting belongs to.
5	**GridSearchCV** tries all 80 combinations with 5-fold cross-validation (400 trainings). **refit='ap'** means the winner is chosen by PR-AUC and retrained on all training data.
6	**.fit** starts the search.
7	Prints the best settings.
8–12	Turns all results into a table, keeps the useful columns, sorts by PR-AUC and shows the top 8.

Result: Best: entropy, depth 10, leaf size 20, balanced weights. PR-AUC 0.500, recall 0.711.

6. Attribute experiments

Tests whether the two extra attributes help, keeping the model fixed so only the attributes change.

Code

```
best9 = {k.replace('tree__', ''): v for k, v in gs9.best_params_.items()}
feature_sets = {'Nine attributes': features9,
                '+ External Destination': features9 + [ED],
                '+ hour': features9 + [HOUR],
                '+ External Destination + hour': ALL}
attr = pd.DataFrame([cv_row(n, tree_pipe(c, **best9), c) for n, c in feature_sets.items()])
attr.to_csv('results/attribute_experiments.csv', index=False)
attr
```

Line by line

Lines	What it does
1	Takes the best settings from Section 5 and removes the **tree__** prefix, so they can be passed to **tree_pipe**.
2–5	Four attribute sets: the nine, nine + External Destination, nine + hour, and all eleven.
6	Evaluates the same tuned tree on each set using **cv_row**. This is a *list comprehension*: a compact loop that builds one result row per set.
7–8	Saves and shows the table. (In Excel, names starting with + display as #NAME?; the numbers are correct.)

Result: PR-AUC rises from 0.500 (nine) to 0.622 (all eleven).

6b. Target-leakage check

Checks that the new attributes are not secretly the answer itself. If an attribute determined the label (e.g. 100% leakage), it would be suspicious.

Code

```
# target-leakage check: both attributes shift the risk but do not determine the label
print(pd.crosstab(X_tr[ED], y_tr, normalize='index').round(3), '\n')
print(pd.crosstab(X_tr[HOUR].between(9, 17).map({True: '09-17', False: 'other'}), y_tr,
                  normalize='index').round(3))
```

Line by line

Lines	What it does
2	Leakage rate for internal and external destinations. **normalize='index'** gives proportions per row.
3–4	Leakage rate during and outside working hours. **.map** renames True/False to readable labels.

Result: External 39.0% vs internal 25.9%; off-hours 40.3% vs working hours 15.8%. The attributes shift the risk but do not decide the label, so they are genuine signals.

7. Comparing models

Compares the decision tree with two stronger but less explainable models, all on the final 11 attributes.

Code

```
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
pos_w = (y_tr == 0).sum() / (y_tr == 1).sum()

def make_pipe(model, cols=ALL):
    return Pipeline([('prep', make_prep(cols)), ('model', model)])

searches = {
    'Decision Tree': (DecisionTreeClassifier(class_weight='balanced', random_state=42),
        {'model__criterion': ['gini', 'entropy'], 'model__max_depth': [7, 10, None],
         'model__min_samples_leaf': [1, 20, 50]}),
    'Random Forest': (RandomForestClassifier(n_estimators=200, class_weight='balanced',
                                             random_state=42, n_jobs=1),
        {'model__max_depth': [10, None], 'model__min_samples_leaf': [1, 20]}),
    'XGBoost': (XGBClassifier(n_estimators=300, scale_pos_weight=pos_w, eval_metric='logloss',
                              random_state=42, n_jobs=1),
        {'model__max_depth': [3, 6], 'model__learning_rate': [0.05, 0.1]}),
}
rows, best = [], {}
for name, (model, grid) in searches.items():
    g = GridSearchCV(make_pipe(model), grid, cv=cv, scoring=scoring, refit='ap', n_jobs=-1)
    g.fit(X_tr[ALL], y_tr)
    i, r = g.best_index_, g.cv_results_
    rows.append({'model': name, **{m: round(r[f'mean_test_{m}'][i], 4) for m in scoring},
                 'params': {k.replace('model__', ''): v for k, v in g.best_params_.items()}})
    best[name] = g
models_cmp = pd.DataFrame(rows)
models_cmp.to_csv('results/model_comparison.csv', index=False)
models_cmp
```

Line by line

Lines	What it does
1–2	Imports **RandomForestClassifier** (many trees voting) and **XGBClassifier** (boosted trees that correct each other's mistakes).
3	**pos_w** = normal events ÷ leakage events (about 2.2). XGBoost uses it to weight leakage events, like class_weight does for the tree.
5–6	**make_pipe**: preparation + any model, so all three models get identical preparation.
8–18	For each model, the model itself and its settings to try. Random Forest uses 200 trees (**n_estimators**). XGBoost uses 300 trees with **learning_rate** controlling how big each correction step is. **n_jobs=1** inside the model avoids overloading the processor, because the search itself already runs in parallel.
19–26	Loops over the three models: tunes each with **GridSearchCV** (chosen by PR-AUC), stores the cross-validation scores of the best setting, and keeps the fitted search in **best** for later.
27–29	Saves and shows the comparison table.

Result: Tree 0.622, Random Forest 0.633, XGBoost 0.634 PR-AUC. Only about 0.01 apart, so the explainable tree is kept.

7b. Imbalance strategies

Compares four ways to stop the model from ignoring the minority (leakage) class.

Code

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTEN, RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler

tree_params = {k.replace('model__', ''): v for k, v in best['Decision Tree'].best_params_.items()}
tree_params['random_state'] = 42
balancers = {'class_weight = balanced': None, 'SMOTEN': SMOTEN(random_state=42),
             'Random oversampling': RandomOverSampler(random_state=42),
             'Random undersampling': RandomUnderSampler(random_state=42)}
rows = []
for name, sampler in balancers.items():
    steps = [('impute', SimpleImputer(strategy='most_frequent').set_output(transform='pandas'))]
    if sampler is not None:
        steps.append(('sampler', sampler))           # training folds only
    cw = 'balanced' if sampler is None else None
    steps += [('prep', make_prep(ALL)), ('tree', DecisionTreeClassifier(class_weight=cw,
    **tree_params))]
    s = cross_validate(ImbPipeline(steps), X_tr[ALL], y_tr, cv=cv, scoring=scoring, n_jobs=-1)
    rows.append({'balancer': name, **{m: round(s[f'test_{m}'].mean(), 4) for m in scoring}})
bal = pd.DataFrame(rows); bal.to_csv('results/imbalance_strategies.csv', index=False)
print('tree:', tree_params); bal
```

Line by line

Lines	What it does
1–3	Imports from **imbalanced-learn**: its own **Pipeline** (needed because samplers change the number of rows), **SMOTEN** (creates synthetic leakage events for categorical data), **RandomOverSampler** (copies leakage events) and **RandomUnderSampler** (removes normal events).
5–6	Takes the best tree settings from Section 7 and fixes the random seed.
7–9	The four strategies. **None** means "no resampling; use class weights instead".
12	First step: fill missing values (samplers cannot handle them). **set_output(transform='pandas')** keeps column names.
13–14	Adds the sampler. Inside the pipeline it only changes the **training folds**, never the fold being evaluated. Resampling before splitting would leak synthetic copies into the evaluation.
15–16	Uses class weights only when no sampler is used, so the two methods are not combined; then adds preparation and the tree.
17–18	Cross-validates each version and stores the averages.
19–20	Saves and shows the table.

Result: Class weights are best (PR-AUC 0.620); SMOTEN is the weakest (0.601) and slowest.

8. Choosing the alarm threshold

The tree outputs a risk score from 0 to 1. The threshold decides above which score an event raises an alarm. It is chosen on training data only.

Code

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import precision_recall_curve
final = best['Decision Tree'].best_estimator_          # refit on all training rows

oof = cross_val_predict(final, X_tr[ALL], y_tr, cv=cv, method='predict_proba', n_jobs=-1)[:, 1]
prec, rec, thr = precision_recall_curve(y_tr, oof)
prec, rec = prec[:-1], rec[:-1]
tab = pd.DataFrame({'threshold': thr, 'precision': prec, 'recall': rec,
                    'f1': 2 * prec * rec / (prec + rec + 1e-12),
                    'alert_rate': [(oof >= t).mean() for t in thr]})

r_star = 0.80
row_f1 = tab.loc[tab.f1.idxmax()]
cand = tab[tab.recall >= r_star]
row_r = cand.loc[cand.precision.idxmax()]
theta = float(row_r.threshold)
print(pd.DataFrame([row_f1, row_r], index=['Maximum F1', f'Most precise with recall ≥
    {r_star}']).round(4))
print('\nchosen θ* =', round(theta, 4))
```

Line by line

Lines	What it does
3	**final** = the best decision tree from Section 7, already retrained on all training data.
5	**cross_val_predict** gives every training event a score from a model that did **not** see that event (out-of-fold scores). **predict_proba** returns the probability of each class; **[:, 1]** keeps the leakage probability.
6	**precision_recall_curve** computes precision and recall for every possible threshold.
7	Drops the last value, which has no matching threshold, so all lists have the same length.
8–10	A table with, for every threshold: precision, recall, F1 (the formula 2 × P × R ÷ (P + R); **1e-12** avoids dividing by zero) and the alert rate (share of events flagged).
12	**r_star = 0.80**: the goal is to catch at least 80% of leaks.
13	For comparison: the threshold with the highest F1.
14–16	Keeps only thresholds with recall ≥ 0.80, then picks the one with the highest precision (fewest false alarms). That threshold is **theta** (θ*).
17–18	Prints both options and the chosen θ*.

Result: θ* = 0.416: recall 0.800, precision 0.508, 49% of events flagged (training estimate).

8b. Operating points

Shows the trade-off between catching leaks and the number of alarms, for different recall targets.

Code

```
# operating points: alerts per 1,000 events at different recall targets (training OOF)
ops = []
for target in [0.5, 0.6, 0.7, 0.8, 0.9]:
    c = tab[tab.recall >= target]; rr = c.loc[c.precision.idxmax()]
    ops.append({'recall target': target, 'threshold': round(rr.threshold, 3),
                'precision': round(rr.precision, 3), 'recall': round(rr.recall, 3),
                'alerts per 1,000 events': int(round(rr.alert_rate * 1000)),
                'true leaks per 1,000 alerts': int(round(rr.precision * 1000))})
ops = pd.DataFrame(ops); ops.to_csv('results/operating_points.csv', index=False); ops
```

Line by line

Lines	What it does
3–4	For each target (50% to 90% of leaks caught), finds the most precise threshold that reaches it.
5–8	Records the threshold, precision, recall, alarms per 1,000 events and real leaks per 1,000 alarms.
9	Saves and shows the table.

Result: At 50% recall about 2 in 3 alarms are real leaks; at 90% recall fewer than 1 in 2.

9. Final test (used once)

Opens the locked test set for the first and only time, and compares three models on it.

Code

```
from sklearn.metrics import (average_precision_score, f1_score, recall_score, precision_score,
                             accuracy_score, matthews_corrcoef, roc_auc_score, confusion_matrix)
ref = baseline.fit(X_tr[features9], y_tr)

def report(name, y_true, score, t):
    pred = (score >= t).astype(int)
    return {'model': name, 'PR-AUC': average_precision_score(y_true, score),
            'ROC-AUC': roc_auc_score(y_true, score) if score.std() > 0 else 0.5,
            'F1': f1_score(y_true, pred), 'Recall': recall_score(y_true, pred),
            'Precision': precision_score(y_true, pred, zero_division=0),
            'Accuracy': accuracy_score(y_true, pred), 'MCC': matthews_corrcoef(y_true, pred),
            'Alert rate': pred.mean()}

s_final = final.predict_proba(X_te[ALL])[:, 1]
s_ref = ref.predict_proba(X_te[features9])[:, 1]
test = pd.DataFrame([report('Majority (always normal)', y_te, np.zeros(len(y_te)), 0.5),
                     report('Reference tree (article)', y_te, s_ref, 0.5),
                     report(f'Final tree (0*={theta:.3f})', y_te, s_final, theta)]).round(4)
test.to_csv('results/test_results.csv', index=False)
cm = confusion_matrix(y_te, (s_final >= theta).astype(int))
print('Final tree confusion matrix [[TN FP] [FN TP]]:\n', cm)
test
```

Line by line

Lines	What it does
1–2	Imports the scoring functions, including **matthews_corrcoef** (MCC: a balanced score from −1 to 1 that stays fair with imbalanced classes) and **roc_auc_score**.
3	Trains the article's reference tree on the training set.
5–12	**report** turns risk scores into yes/no decisions with threshold **t** (**score >= t**), then computes all scores. ROC-AUC is set to 0.5 when all scores are identical, which happens for the "always normal" model.
14–15	Risk scores on the test set from the final tree (11 attributes) and the reference tree (9 attributes).

Lines	What it does
16–18	Evaluates three models: always "normal" (a score of 0 for every event), the reference tree at the usual threshold 0.5, and the final tree at θ*.
19–21	Saves the table and prints the confusion matrix: correct normals (TN), false alarms (FP), missed leaks (FN) and caught leaks (TP).
22	Shows the table.

Result: Final tree: recall 0.827, precision 0.507, PR-AUC 0.609, MCC 0.429. Reference tree: recall 0.101, MCC 0.127.

9b. Test charts

Draws the confusion matrix and the precision-recall curves.

Code

```
from sklearn.metrics import ConfusionMatrixDisplay, PrecisionRecallDisplay
fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
ConfusionMatrixDisplay(cm, display_labels=['Normal', 'Leakage']).plot(ax=axes[0], colorbar=False)
axes[0].set_title(f'Final tree, θ* = {theta:.3f}')
PrecisionRecallDisplay.from_predictions(y_te, s_final, ax=axes[1], name='Final tree')
PrecisionRecallDisplay.from_predictions(y_te, s_ref, ax=axes[1], name='Reference tree')
p_ = precision_score(y_te, s_final >= theta); r_ = recall_score(y_te, s_final >= theta)
axes[1].scatter([r_], [p_], color='red', zorder=5, label='chosen threshold'); axes[1].legend()
axes[1].set_title('Precision-recall, test set')
plt.tight_layout(); plt.savefig('results/test_evaluation.png', dpi=150, bbox_inches='tight'); plt.show()
```

Line by line

Lines	What it does
2	Creates a figure with two side-by-side panels (**axes[0]** and **axes[1]**).
3–4	Left panel: the confusion matrix as a coloured grid.
5–6	Right panel: precision-recall curves of both trees. A curve further up and to the right is better.
7–8	Marks the chosen threshold as a red dot.
10	Saves and shows the figure.

Result: results/test_evaluation.png

10. IF-THEN rules

Turns the tree into readable rules and finds the five rules that catch the most leakage. This is the most technical cell; the idea is simple: follow each path from the top of the tree to a final group (leaf) and write the questions down.

Code

```
from sklearn.inspection import permutation_importance
prep, tree = final.named_steps['prep'], final.named_steps['model']
names = prep.get_feature_names_out()
enc = prep.named_transformers_['cat'].named_steps['onehot']
cat_cols = prep.transformers_[0][2]
cats = {col: [str(v) for v in vals] for col, vals in zip(cat_cols, enc.categories_)}
t = tree.tree_

def split(i):
    nm = names[t.feature[i]]
    if nm == HOUR:
        return HOUR, None
    col = max((c for c in cats if nm.startswith(c + '_')), key=len)
    return col, nm[len(col) + 1:]

paths = {}
def walk(node, conds):
```

```
    if t.children_left[node] == -1:
        paths[node] = conds; return
    col, val = split(node)
    if col == HOUR:
        walk(t.children_left[node], conds + [(HOUR, '≤', t.threshold[node])])
        walk(t.children_right[node], conds + [(HOUR, '>', t.threshold[node])])
    else:
        walk(t.children_left[node], conds + [(col, '≠', val)])
        walk(t.children_right[node], conds + [(col, '=', val)])
walk(0, [])

def pretty(v):
    return {'1.0': '1', '0.0': '0'}.get(v, v)

def readable(conds):
    eq, excl, order, lo, hi = {}, {}, [], None, None
    for c, op, v in conds:
        if c == HOUR:
            if op == '≤': hi = v if hi is None else min(hi, v)
            else: lo = v if lo is None else max(lo, v)
            continue
        if c not in order: order.append(c)
        if op == '=': eq[c] = v
        else: excl.setdefault(c, set()).add(v)
    parts = []
    for c in order:
        if c in eq:
            parts.append(f'{c} = {pretty(eq[c])}')
        else:
            left = [pretty(o) for o in cats[c] if o not in excl[c] and o != 'nan']
            parts.append(f'{c} = {left[0]}' if len(left) == 1 else f"{c} in {{{', '.join(left)}}}")
    if lo is not None or hi is not None:
        a = 0 if lo is None else int(np.floor(lo)) + 1
        b = 23 if hi is None else int(np.floor(hi))
        parts.append(f'hour {a:02d}:00-{b:02d}:59')
    return ' AND '.join(parts)

s_tr = final.predict_proba(X_tr[ALL])[:, 1]
leaf = tree.apply(prep.transform(X_tr[ALL]))
st = (pd.DataFrame({'leaf': leaf, 'y': y_tr.values, 's': s_tr}).groupby('leaf')
        .agg(n=('y', 'size'), leak_rate=('y', 'mean'), score=('s', 'first')))
st['caught'] = (st.n * st.leak_rate).round().astype(int)
alerts = st[st.score >= theta].sort_values('caught', ascending=False)
print(f'Leaves: {len(st)} | leaves raising an alert at θ*: {len(alerts)}\n')
rules = []
for k, (lf, row) in enumerate(alerts.head(5).iterrows(), 1):
    rule = readable(paths[lf]); rules.append({'rule': k, 'IF': rule, 'events': int(row.n),
                                              'leakage rate': round(row.leak_rate, 2), 'caught':
    int(row.caught)})
    print(f'Rule {k}: IF {rule} THEN leakage')
    print(f'   events={int(row.n)}, leakage rate={row.leak_rate:.2f}, caught={int(row.caught)}\n')
pd.DataFrame(rules).to_csv('results/top_rules.csv', index=False)
print('Top 5 rules cover', round(alerts.head(5).caught.sum() / y_tr.sum(), 3), 'of training leakage
    events')
```

Line by line

Lines	What it does
1	Imports permutation importance (used in the next cell).
2	Takes the two parts out of the final pipeline: the preparation step and the tree.
3	**names**: the column names after one-hot encoding, e.g. "External Destination_external".
4–6	**cats**: for each attribute, its possible values, taken from the encoder. Needed to rebuild readable names.
7	**t** = the tree's internal structure (its nodes, splits and children).
9–14	**split(i)**: for node **i**, finds which attribute and value it asks about. For one-hot columns, it splits "Authority_manager" back into attribute "Authority" and value "manager".

Lines	What it does
16–27	**walk** visits the tree from the top (node 0). At each node it records the question and goes left and right. For a one-hot column, left means "≠ value" and right means "= value"; for hour, left means "≤ threshold" and right "> threshold". At a leaf (**children_left == -1**) it stores the full list of conditions in **paths**. A function that calls itself like this is called *recursive*.
29–30	**pretty** shows 1.0 and 0.0 as 1 and 0.
32–53	**readable** simplifies a path into one sentence. It keeps the tightest hour limits and writes them as a time range (e.g. 00:00–07:59). For attributes with only "≠" conditions, it lists the values that remain (e.g. "Authority in {intern, staff}"). The conditions are joined with AND.
55	Risk scores of the final tree for all training events.
56	**tree.apply** tells which leaf each training event ends up in.
57–58	For each leaf: number of events, leakage rate, and its risk score.
59	**caught** = how many leakage events fall in that leaf.
60	Keeps leaves that raise an alarm (score ≥ θ*), sorted by leaks caught.
61–67	Prints the five strongest rules with their size, leakage rate and leaks caught.
68–69	Saves the rules and prints the share of all training leakage they cover.

Result: 545 leaves, 363 raise an alarm. All top 5 rules start with confidential file transfer to an external destination; they cover 7% of training leakage.

10b. Permutation importance

Measures how much each attribute matters: shuffle one attribute's values randomly and see how much performance drops. A big drop means the model relies on it.

Code

```
pi = permutation_importance(final, X_te[ALL], y_te, scoring='average_precision',
                            n_repeats=10, random_state=42, n_jobs=-1)
imp = pd.DataFrame({'mean': pi.importances_mean, 'std': pi.importances_std},
                   index=ALL).sort_values('mean', ascending=False)
imp.to_csv('results/permutation_importance.csv')
imp['mean'].iloc[::-1].plot(kind='barh', xerr=imp['std'].iloc[::-1], figsize=(7, 4), color='#4C72B0')
plt.title('Permutation importance (drop in PR-AUC, test set)'); plt.xlabel('drop in PR-AUC')
plt.savefig('results/permutation_importance.png', dpi=150, bbox_inches='tight'); plt.show()
imp.round(4)
```

Line by line

Lines	What it does
1–2	**permutation_importance** shuffles each of the 11 attributes 10 times (**n_repeats=10**) on the test set and measures the drop in PR-AUC. It is more reliable than the tree's built-in importance because it measures actual effect on performance.
3–4	Builds a table of the average drop and its variation (**std**), sorted from most to least important.
5–9	Saves the table, draws a horizontal bar chart with error bars (**xerr**), saves the chart, and shows the table.

Result: Top: Confidential Data Access 0.134, External Destination 0.133, Confidential File Transfer 0.109, hour 0.103. Data Modification: 0.

11. Saving the model

Saves everything needed to reuse the model and reproduce the run.

Code

```
import joblib
joblib.dump({'model': final, 'threshold': theta, 'features': ALL}, 'models/final_tree.joblib')
run = {'python': sys.version.split()[0], 'sklearn': sklearn.__version__, 'imblearn':
    imblearn.__version__,
       'xgboost': xgboost.__version__, 'random_state': 42, 'test_size': 0.2, 'features': ALL,
       'tree_params': {k: (v if v is not None else 'None') for k, v in tree_params.items()},
       'threshold': theta, 'recall_target': r_star}
json.dump(run, open('results/run_info.json', 'w'), indent=2)
if IN_COLAB:   # download everything as one zip
    !zip -qr dlp_results.zip results models
    from google.colab import files; files.download('dlp_results.zip')
run
```

Line by line

Lines	What it does
1–2	**joblib.dump** saves the final pipeline (preparation + tree), the threshold and the feature list into one file. It can later be loaded to score new events, for example in a cloud API.
3–7	Saves a **run record** (JSON): library versions, random seed, test size, features, tree settings and threshold.
8–10	Only in Colab: zips the **results** and **models** folders and downloads the zip to your computer.
11	Shows the run record.

Result: models/final_tree.joblib and results/run_info.json